

EcoGo

Travel Green · Live Clean

Progetto per il corso di Applicazioni e Servizi Web

Guo Jiahao

Matricola: 0001173814 jiahao.guo@studio.unibo.it

Kalile Horri

Matricola: 0001192440 kalile.horri@studio.unibo.it

Intisar Bel Meddeb

Matricola: 0001186166 Intisar.belmeddeb@studio.unibo.it

5 febbraio 2026

Indice

1	Introduzione	3
1.1	Nota sul Progetto Integrato	3
1.2	Ambientazione dell'elaborato	3
2	Requisiti	4
2.1	Bisogni emersi dal processo di Needfinding	4
2.2	Task principali	4
2.3	Requisiti di business	4
2.4	Requisiti utente	5
2.5	Requisiti funzionali	5
2.6	Requisiti non funzionali	5
3	Design	7
3.1	Scelta architetturale	7
3.2	Stack tecnologico	7
3.3	Metodologie di sviluppo	8
3.4	Strumenti e gestione del flusso di lavoro	8
3.5	Prototipazione e User Interface Design	8
3.6	Interfaccia Utente	10
3.6.1	Storyboard	10
3.6.2	Low Fidelity prototype	11
3.6.3	Medium Fidelity prototype	12
3.6.4	High Fidelity prototype	15
3.7	Target User Analysis	20
3.8	Gamification	22

4	Tecnologie	23
4.1	Pinia	23
4.2	Mapbox GL	23
4.3	Turf.js	23
4.4	Socket.io	23
5	Codice e Funzionalità	24
5.1	Gestione delle chiamate llm	24
5.2	Gestione comunicazione real-time con Socket.io	25
5.3	Gestione stato con Pinia	27
6	Test	29
6.1	Risultati e Miglioramenti	30
7	Deployment	31
7.1	Installazione	31
7.2	Messa in funzione	31
8	Conclusioni	32

1 Introduzione

1.1 Nota sul Progetto Integrato

Questo elaborato presenta i risultati di un progetto integrato sviluppato congiuntamente per i corsi di *Applicazioni e Servizi Web* e *Human-Computer Interaction (HCI)*. L'obiettivo è stato quello di coniugare la realizzazione di un'architettura software robusta e moderna (focus del corso Web) con le metodologie di design centrato sull'utente (focus del corso HCI). Di conseguenza, il documento illustra sia lo stack tecnologico e l'implementazione del codice, sia l'intero ciclo di progettazione dell'esperienza utente, che include l'analisi dei bisogni (Needfinding), la prototipazione iterativa (Low-Fidelity e High-Fidelity) e i test di usabilità.

1.2 Ambientazione dell'elaborato

Il settore del turismo sostenibile affronta due problematiche principali: gli utenti desiderano agire in modo sostenibile ma trovano le opzioni "green" scomode o difficili da reperire, e le decisioni di viaggio sono spesso guidate esclusivamente da costi e comodità piuttosto che dall'impatto ambientale.

Per risolvere questi problemi è stato sviluppato **EcoGo**, un web app di viaggio intelligente. La piattaforma affronta queste sfide attraverso:

- **Facilità di pianificazione:** evidenzia percorsi a basso impatto e supporta il confronto tra alternative (es. treno vs aereo), aiutando l'utente a trovare scelte più sostenibili.
- **Gamification:** utilizza punti, classifiche, obiettivi e suggerimenti basati su AI per rendere il viaggio sostenibile motivante e gratificante.
- **Feedback:** Pagina dedicata al feedback dove gli utenti possono lasciare recensioni, segnalare problemi, suggerire miglioramenti o riportare utenti abusivi.
- **Discover:** Sezione in cui gli utenti condividono le travel cards con la community.

La Value Proposition del progetto è riassunta nel motto: *"Travel Green · Live Clean"*.

2 Requisiti

2.1 Bisogni emersi dal processo di Needfinding

Durante la fase preliminare di analisi dei bisogni, condotta anche attraverso interviste a diversi utenti, sono stati identificati cinque bisogni fondamentali che il sistema deve soddisfare:

1. **Semplicità:** le scelte ecologiche devono essere facili da trovare.
2. **Controllo:** gli utenti necessitano di informazioni chiare per sentirsi sicuri.
3. **Trasparenza:** fiducia nell'uso dei propri dati e nel calcolo delle emissioni.
4. **Autenticità:** desiderio di esperienze locali reali, non trappole per turisti.
5. **Motivazione:** interesse verso competizioni amichevoli e non stressanti.

2.2 Task principali

Sulla base dei bisogni identificati, l'interazione utente è stata strutturata attorno a tre macro-attività (task) principali, graduate per livello di complessità:

- **Discover (Semplice):** trovare rapidamente opzioni di viaggio sostenibili e suggerimenti AI.
- **Plan (Moderato):** organizzare un viaggio bilanciando budget, divertimento e sostenibilità, confrontando le emissioni (es. treno vs auto).
- **Track (Complesso):** monitorare l'impatto (CO_2 risparmiata), visualizzare l'Eco-Score e condividere esperienze.

2.3 Requisiti di business

Gli obiettivi strategici che guidano lo sviluppo della piattaforma includono:

- **Soluzione Integrata:** Offrire una piattaforma end-to-end che accompagni l'utente dalla pianificazione al tracciamento del viaggio sostenibile.
- **Consapevolezza Ambientale:** Rendere tangibile l'impatto delle scelte di mobilità attraverso metriche chiare (CO_2 , eco-score), incentivando l'adozione di comportamenti alternativi.
- **Retention degli Utenti:** Favorire la fidelizzazione e l'uso ricorrente dell'applicazione attraverso meccaniche di gamification, quali l'assegnazione di punti, l'avanzamento di livello e l'ottenimento di badge.
- **Gestione e Monitoraggio:** Predisporre strumenti adeguati per la raccolta di feedback e un pannello di amministrazione per la supervisione dei contenuti e della community.

2.4 Requisiti utente

Dal punto di vista dell'utente finale, il sistema deve garantire:

- **Visualizzazione Immediata dei Dati:** Accesso rapido alle informazioni chiave tramite dashboard, inclusi l'impatto personale, i viaggi attuali, passati e i progressi realizzati.
- **Pianificazione Interattiva:** Supporto visivo e guidato nella creazione dell'itinerario, con l'ausilio di mappe, gestione dei segmenti di viaggio e strumenti di confronto delle alternative.
- **Informazioni Contestuali:** Ricezione di notifiche utili e tempestive nella sezione "Live" (es. aggiornamenti meteo o traffico).
- **User accessibility** Scelta libera nella gestione delle preferenze, come dark theme e lingua del sito.
- **User Experience:** Un'esperienza d'uso fluida e intuitiva, caratterizzata da un'interfaccia a card e feedback immediati ad ogni interazione.

2.5 Requisiti funzionali

Il sistema dovrà implementare le seguenti funzionalità tecniche:

- **Autenticazione** (JWT) e gestione sessione lato client.
- **Profilo:** aggiornamento dati, cambio password, preferenze.
- **Pianificazione su Mappa:** Strumenti per la creazione, modifica e salvataggio di segmenti di viaggio, con calcolo integrato delle emissioni per il confronto tra mezzi.
- **Dashboard:** Visualizzazione di grafici relativi alle performance ambientali (CO₂ risparmiata, distanze, eco-score, etc...).
- **Rewards e Gamification:** Logica di backend per l'assegnazione di punti, calcolo del livello utente, gestione degli achievements e classifiche.
- **Integrazione AI:** Servizi di IA (es. Gemini) al fine di generare consigli di viaggio e contenuti educativi.
- **Notifiche real-time:** ricezione eventi via Socket.io.
- **Amministrazione:** Pannello di controllo per la gestione e supervisione della piattaforma (feedback, utenti, contenuti).

2.6 Requisiti non funzionali

Per garantire standard qualitativi, il sistema rispetterà i seguenti vincoli:

- **Sicurezza:** password hashate (bcrypt), JWT, CORS, validazione input.
- **Affidabilità:** persistenza dati su MongoDB (volume Docker in locale).

- **Manutenibilità:** separazione client/server, modularità in routes/controllers/services.
- **Performance:** caricamenti progressivi, richieste API ottimizzate, componentizzazione UI.
- **Indipendenza:** containerizzazione con Docker

3 Design

3.1 Scelta architetturale

Per la progettazione e lo sviluppo del sistema, la scelta è ricaduta sull'adozione di un'architettura basata su **Web Application**. Questa decisione strategica risponde alla necessità di creare un sito flessibile e universalmente accessibile per l'utente finale.

A differenza di soluzioni software native che richiedono installazioni specifiche, il formato Web Application consente agli utenti di fruire delle funzionalità core della piattaforma — quali la pianificazione dettagliata dei viaggi e la visualizzazione analitica della propria impronta di carbonio — in modo immediato e trasparente.

I vantaggi principali di questa impostazione architetturale includono:

- **Accessibilità:** La piattaforma è fruibile da diversi dispositivi (desktop, tablet, smartphone) dotato di un browser moderno e di una connessione a internet, garantendo la continuità dell'uso.
- **Indipendenza dall'Hardware:** Non è richiesto alcun hardware specializzato né risorse computazionali elevate lato client, lasciando il carico di elaborazione al server.
- **Manutenibilità:** La natura centralizzata della Web Application permette di distribuire aggiornamenti e nuove funzionalità facilmente a tutti gli utenti, senza richiedere azioni manuali di aggiornamento sui singoli dispositivi.

3.2 Stack tecnologico

L'architettura del sistema si basa su **MEVN**, separando il frontend dal backend per garantire modularità e scalabilità.

- **Client:** L'interfaccia utente è stata sviluppata utilizzando il framework **Vue 3**, supportato dal build tool **Vite** per ottimizzare le prestazioni di sviluppo e produzione. La gestione della logica applicativa include il routing, l'architettura a componenti e la gestione dello stato tramite **Pinia**. Il client gestisce la comunicazione bidirezionale con il server attraverso chiamate API asincrone e connessioni WebSocket.
- **Server:** La logica di business risiede su un server **Node.js** equipaggiato con il framework **Express**. L'architettura è strutturata per esporre API RESTful organizzate in controller e servizi modulari, inclusa l'integrazione con servizi esterni (come le API di Gemini per l'intelligenza artificiale). Il server gestisce inoltre le notifiche push tramite utilizzo Socket.io.
- **Database:** Per la memorizzazione dei dati è stato scelto il database NoSQL **MongoDB**, ideale per la sua flessibilità nella gestione di documenti JSON-like. L'interazione con il database è mediata dalla libreria **Mongoose**, che definisce gli schemi per le principali collezioni del sistema: *users*, *trips*, *notifications*, *feedback*, *weather*, *achievements* e *travelcards*.

3.3 Metodologie di sviluppo

Per la gestione del progetto e la realizzazione dell'elaborato è stata adottata la metodologia di sviluppo **Agile**. Questa scelta è stata dettata dalla necessità di mantenere un approccio flessibile e iterativo, permettendo al gruppo di adattare i requisiti sulla base dei feedback e di rilasciare funzionalità in modo incrementale. L'approccio Agile è stato integrato con i principi dello **User Centered Design**, ponendo l'utente e i suoi bisogni (identificati nella fase di needfinding e durante il corso di Hci) come nucleo delle decisioni.

Per massimizzare la User Experience e l'usabilità dell'applicazione, lo sviluppo ha seguito tre linee guida fondamentali:

- **Design Consistency:** è stato utilizzato un design uniforme per colori, icone e caratteri, così da rendere l'esperienza più coerente in tutte le sezioni dell'app (Discover, Plan, Dashboard, ecc.) e facilitare l'apprendimento da parte degli utenti.
- **Responsive Design:** L'interfaccia è stata realizzata per adattarsi a diverse dimensioni di schermo, garantendo accessibilità da qualsiasi dispositivo.
- **KISS:** Le interfacce sono state progettate per essere chiare e intuitive, così da permettere all'utente di muoversi tra le pagine in modo facile e immediato.

3.4 Strumenti e gestione del flusso di lavoro

L'organizzazione del progetto è stata supportata dall'uso di strumenti dedicati al design e alla gestione delle attività:

- **Prototipazione con Figma:** Prima di scrivere il codice, ogni funzionalità è stata progettata a livello visivo tramite mockup e prototipi interattivi realizzati con Figma. Questo ha permesso di verificare in anticipo il design e i flussi di navigazione, come descritto nel paragrafo successivo.
- **Gestione dei Task con Trello:** Per il tracciamento delle attività è stata utilizzata la piattaforma Trello, tramite una board condivisa che ha permesso di monitorare lo stato di avanzamento dello sviluppo e coordinare in modo efficace l'implementazione parallela delle componenti client e server.

3.5 Prototipazione e User Interface Design

Il processo di design ha seguito un approccio incrementale, suddiviso in più fasi con un livello di dettaglio progressivamente crescente, come previsto dai principi del corso di Human-Computer Interaction

1. **Storyboard e Flusso Narrativo:** In una prima fase è stato definito il flusso logico dell'esperienza utente tramite una storyboard, che ha permesso di visualizzare la sequenza delle azioni (Discover, Plan, Dashboard) prima di passare alla definizione grafica delle schermate.
2. **Low-Fidelity (Paper Prototyping):** La rappresentazione visiva è stata realizzata tramite schizzi a mano libera, utili per definire rapidamente la disposizione generale degli elementi e la struttura di navigazione, senza concentrarsi sui dettagli estetici.

3. **Medium-Fidelity (Digital Prototyping):** I concetti sviluppati su carta sono stati poi digitalizzati tramite il software Figma.
4. **High-Fidelity:** Questo livello è stato realizzato direttamente scrivendo il codice. Partendo dal prototipo a media fedeltà, l'interfaccia finale è stata costruita e perfezionata nell'ambiente di sviluppo, definendo dettagli informativi, interazioni e reattività del sistema.

3.6 Interfaccia Utente

Il design visivo utilizza una palette ispirata alla natura con tonalità predominanti Smeraldo , Verde Foresta e un layout basato su *card* per rendere i dati complessi facili da leggere e scrivere. Di seguito viene mostrata l'evoluzione dell'interfaccia attraverso le diverse fasi di progettazione.

3.6.1 Storyboard

Esempio di scenario:

1. L'utente effettua il login e consulta la Dashboard.
2. Pianifica un nuovo viaggio nella sezione Plan.
3. Salva il viaggio e visualizza CO₂ stimata ed Eco-Score.
4. Consulta il forum integrato per scoprire punti di interesse.
5. Riceve una notifica e consulta i consigli per migliorare.
6. Sblocca un achievement nella sezione di gamification.



Figura 1: Storyboard: Flusso narrativo preliminare.

3.6.2 Low Fidelity prototype

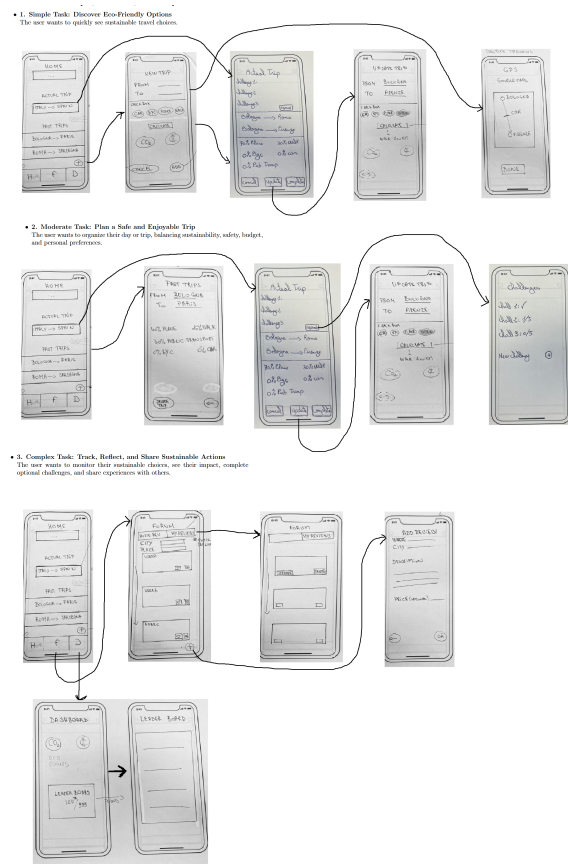


Figura 2: Low-Fidelity: Schizzi su carta delle interfacce principali.

3.6.3 Medium Fidelity prototype

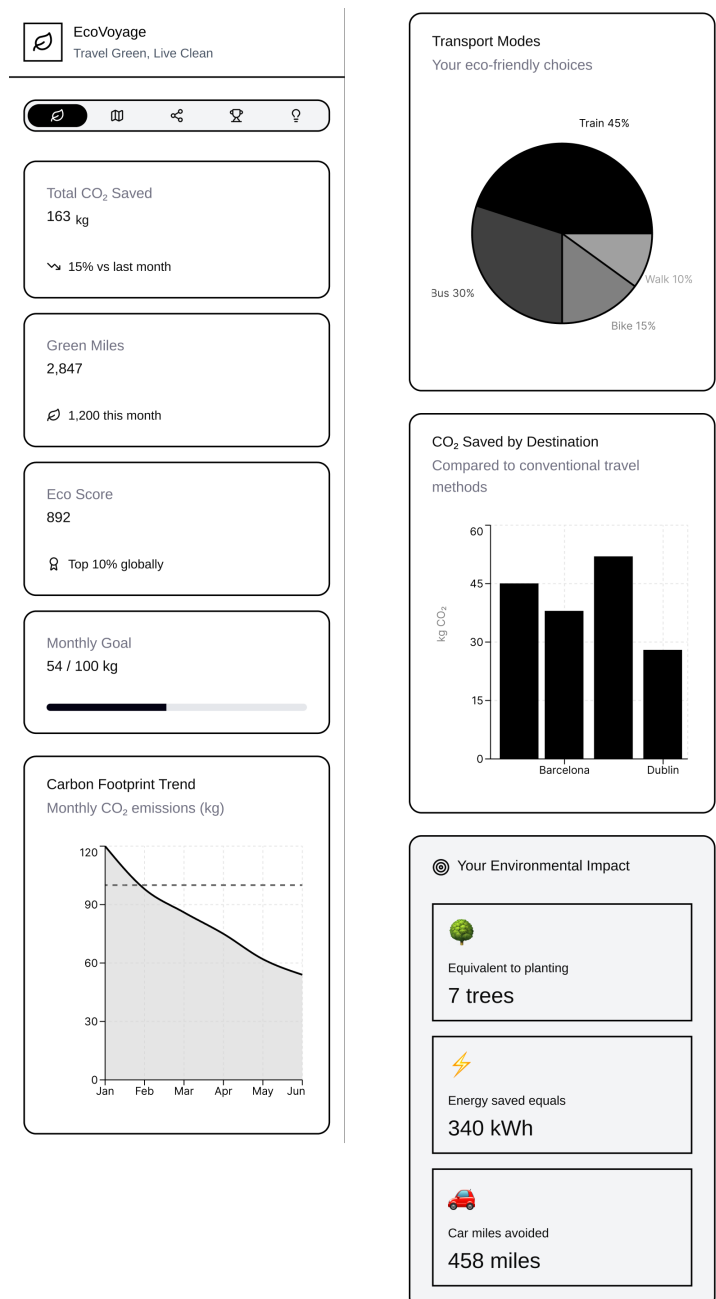


Figura 3: Medium-Fidelity prototype 1



EcoVoyage
Travel Green, Live Clean



Plan Your Green Journey

Calculate the carbon footprint of your trip

📍 From

📍 To

Transport Mode



Distance (km)

👤 Travelers



Calculate Carbon Footprint



Enter your travel details to calculate
your carbon footprint

Figura 4: Medium-Fidelity prototype 2.



Figura 5: Medium-Fidelity prototype 3.

3.6.4 High Fidelity prototype

Infine, le schermate del **prototipo High-Fidelity**, che mostrano il design visivo definitivo comprensivo di componenti e dettagli stilistici.

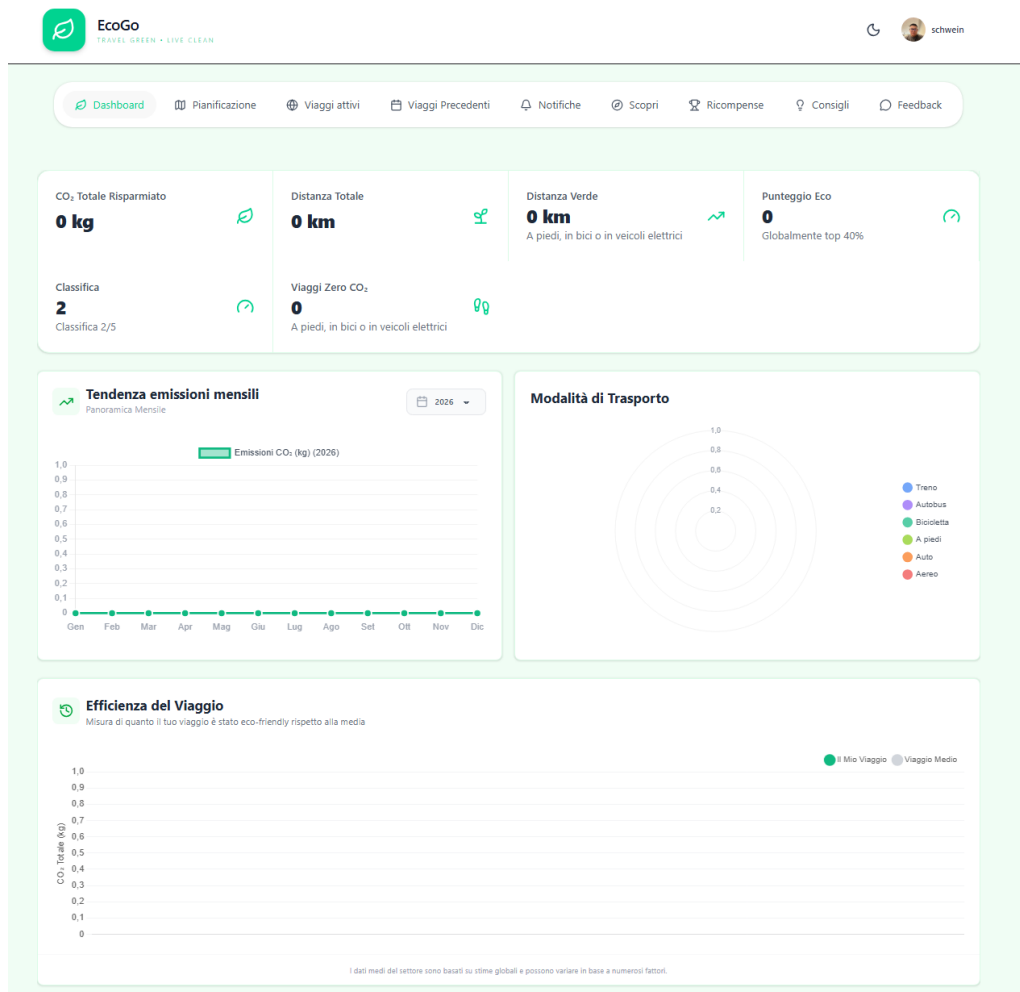


Figura 6: High-Fidelity: Dashboard.

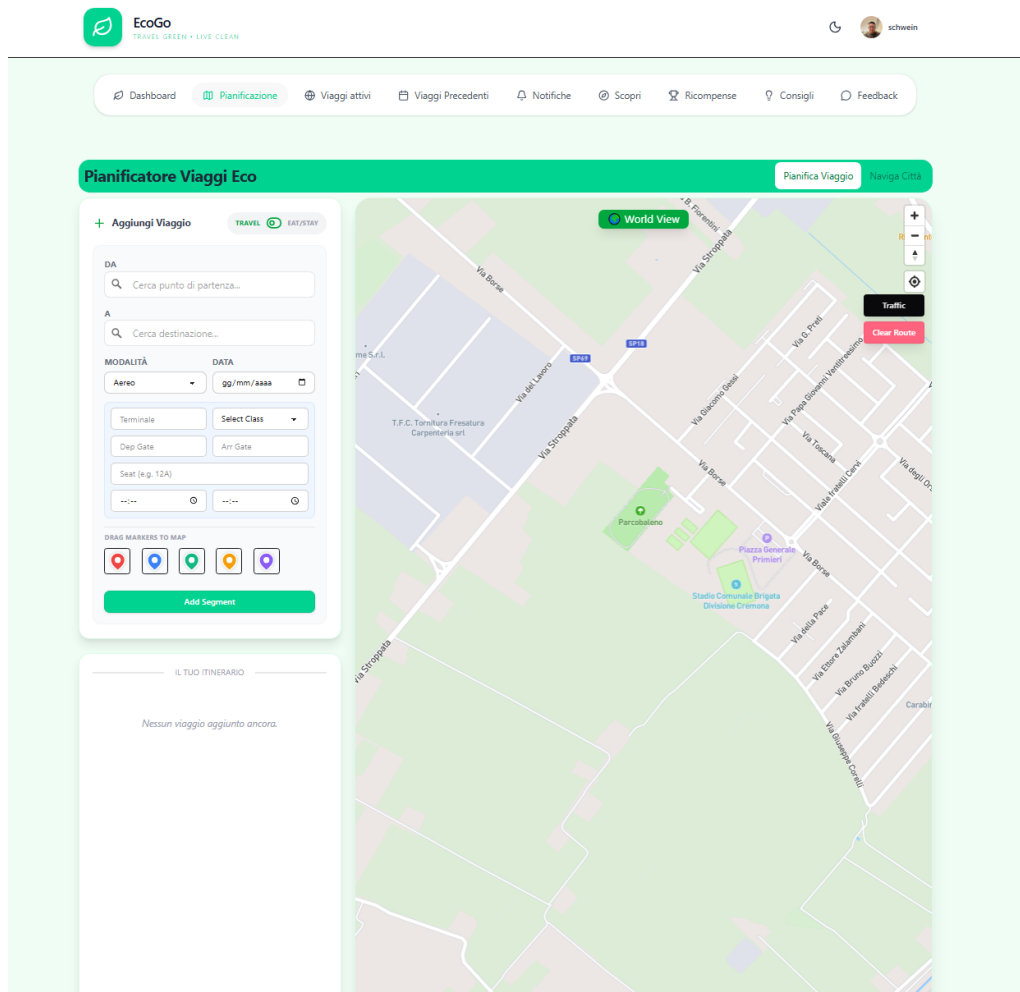


Figura 7: High-Fidelity: Plan.

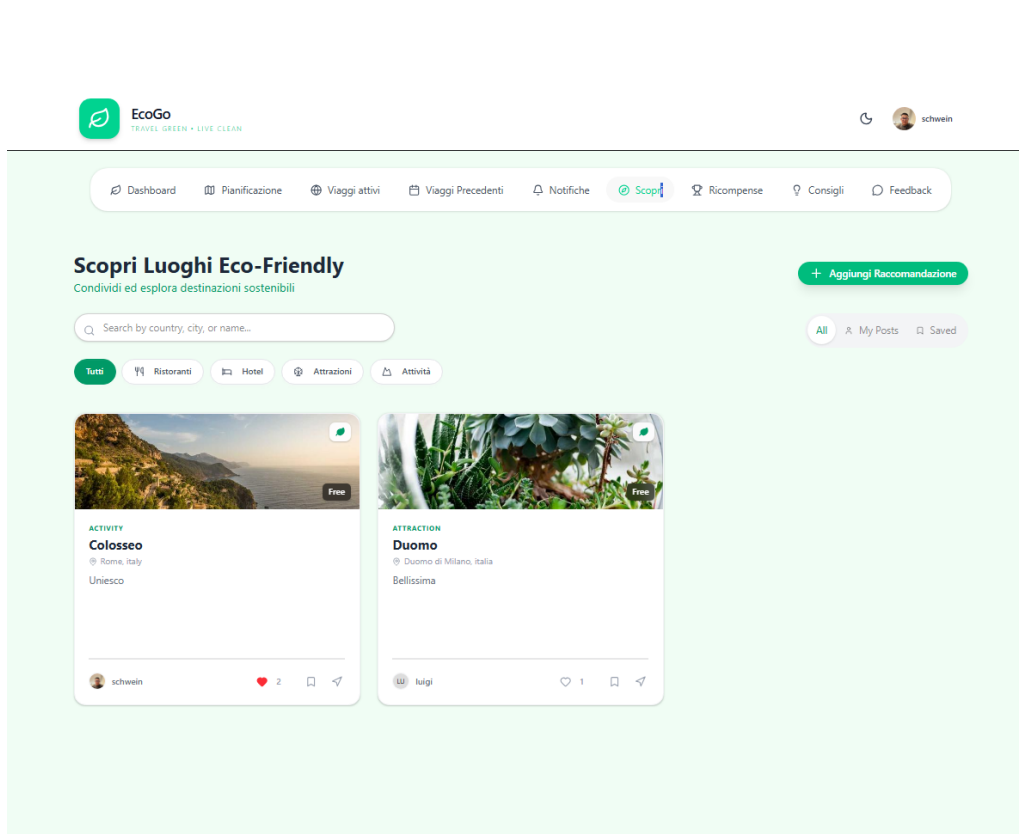


Figura 8: High-Fidelity: Discover.

EcoGo

TRAVEL GREEN • LIVE CLEAN

Dashboard

Planificazione

Viaggi attivi

Viaggi Precedenti

Notifiche

Scopri

Ricompense

Consigli

Feedback

Valutazione Media Utente

★ 2.2 / 5

Feedback Totali

9

Tasso di Implementazione

0%

Invia Feedback

Aiutaci a migliorare EcoGo con i tuoi suggerimenti

Categoria

Seleziona categoria

La Tua Valutazione

☆☆☆☆☆

Oggetto

Breve descrizione del tuo feedback

Messaggio

Fornisci feedback dettagliato...

Invia Feedback

Feedback della Comunità

Sfoglia e vota sui feedback di altri utenti

Mi piace questo sito

Nuovo

schwein

3 hours ago

★★★★★

No additional comments provided.

Vota (0)

Feedback Generale

eifzknje

Nuovo

kali horri

giorno fa

★★★☆☆

ezfze

Vota (0)

Richiesta Funzionalità

Figura 9: High-Fidelity: Feedback.

18

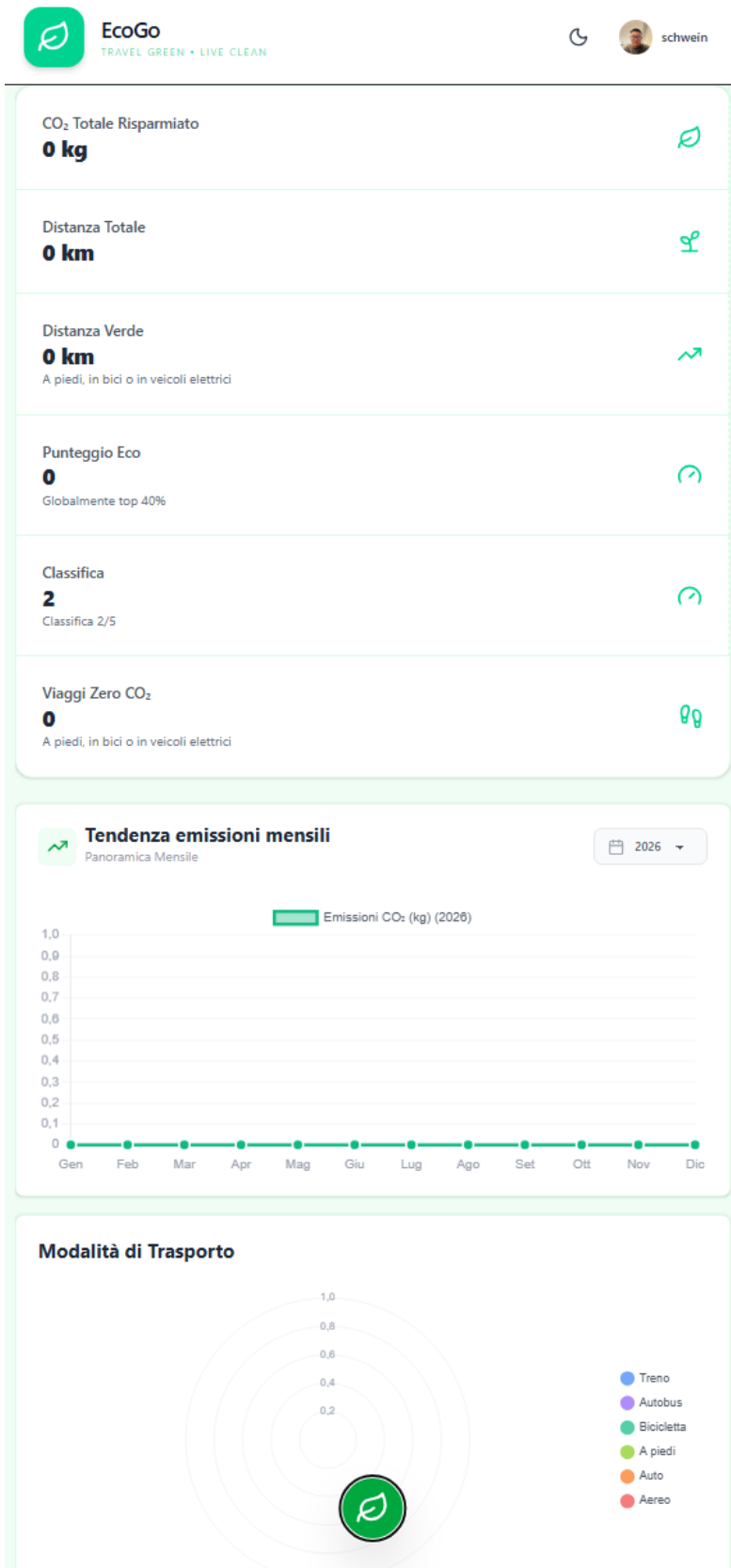


Figura 10: High-Fidelity: Responsive version

3.7 Target User Analysis

Per validare le ipotesi iniziali e raccogliere dati qualitativi, è stato scelto un campione di quattro partecipanti. La selezione ha considerato diverse tipologie di utenti, dai “Immediate Users” (utenti target standard) agli “Extreme Users” (utenti con esigenze o comportamenti particolari), per ottenere una visione completa delle abitudini di viaggio. Di seguito sono riportati i profili dei partecipanti coinvolti:

Partecipante 1: Carafassi Samuele (Immediate User)

Studente di Cybersecurity presso l’Università di Padova.

- **Motivazione della scelta:** In quanto studente universitario, rappresenta perfettamente il target primario. Possiede una naturale predisposizione all’uso della tecnologia e delle applicazioni di viaggio.
- **Pertinenza:** Incarna lo stile di vita dell’utente giovane e attivo.
- **Reclutamento:** Contattato tramite il network universitario (Università di Padova) previo consenso informato sugli scopi dello studio.

Partecipante 2: Farinelli Simone (Immediate User)

Studente di Game Development e Design presso l’Università di Bristol.

- **Motivazione della scelta:** Risiedendo all’estero, viaggia con alta frequenza.
- **Pertinenza:** Può fornire feedback preziosi sia dal punto di vista dell’esperienza utente in mobilità internazionale, sia, data la sua formazione, sulle meccaniche di gamification.
- **Reclutamento:** Contattato tramite l’Università di Bristol previo consenso informato.

Partecipante 3: Isa Yaqoubi (Extreme User)

Studente di Ingegneria Biomedica presso l’Università di Bologna.

- **Motivazione della scelta:** Approccio dinamico alla mobilità e forte familiarità con le app di viaggio.
- **Pertinenza:** Classificato come "Extreme User" per la sua attitudine critica verso gli strumenti digitali di mobilità.
- **Reclutamento:** Selezione tramite raccomandazione diretta.

Partecipante 4: Ramzy Cherif (Immediate User)

Neolaureato, impiegato presso l’Ufficio Nazionale Erasmus+.

- **Motivazione della scelta:** La sua occupazione lo pone in costante contatto con studenti in mobilità internazionale, offrendo un punto di vista avvantaggiato sulle loro esigenze.
- **Pertinenza:** Fornisce feedback rilevanti non solo come viaggiatore, ma anche come osservatore.
- **Reclutamento:** Contattato tramite network personale.

L’analisi dei potenziali utenti del sito ha portato alla definizione di tre profili (*Personas*):

- **Viaggiatori occasionali:** vogliono capire l'impatto senza complessità e trovare alternative veloci.
- **Viaggiatori frequenti:** vogliono confrontare opzioni e monitorare progressi nel tempo.
- **Utenti eco-engaged:** cercano dettagli, obiettivi, badge e motivazione tramite ranking.

3.8 Gamification

Per rendere accattivante e premiare scelta di mezzi di trasporto per un viaggio verde, EcoGo integra un sistema di gamification leggera, progettato per motivare l'utente senza risultare invasivo o stressante. Le meccaniche implementate includono:

- **Sistema di Progressione:** Ogni scelta sostenibile fa guadagnare Eco-Points. Accumulando punti, l'utente può salire di livello, ottenendo una rappresentazione concreta del proprio contributo ecologico e valorizzando l'impegno costante.
- **Achievements:** Sono stati introdotti obiettivi specifici per diversi tipi di interazione, come il raggiungimento di un certo numero di viaggi totali o la scelta frequente di opzioni a basso impatto.
- **Leaderboard:** Per stimolare la motivazione sociale, è stata introdotta una classifica che crea una competizione amichevole. Gli utenti possono confrontare i propri risultati con quelli della community, incoraggiando un miglioramento continuo.

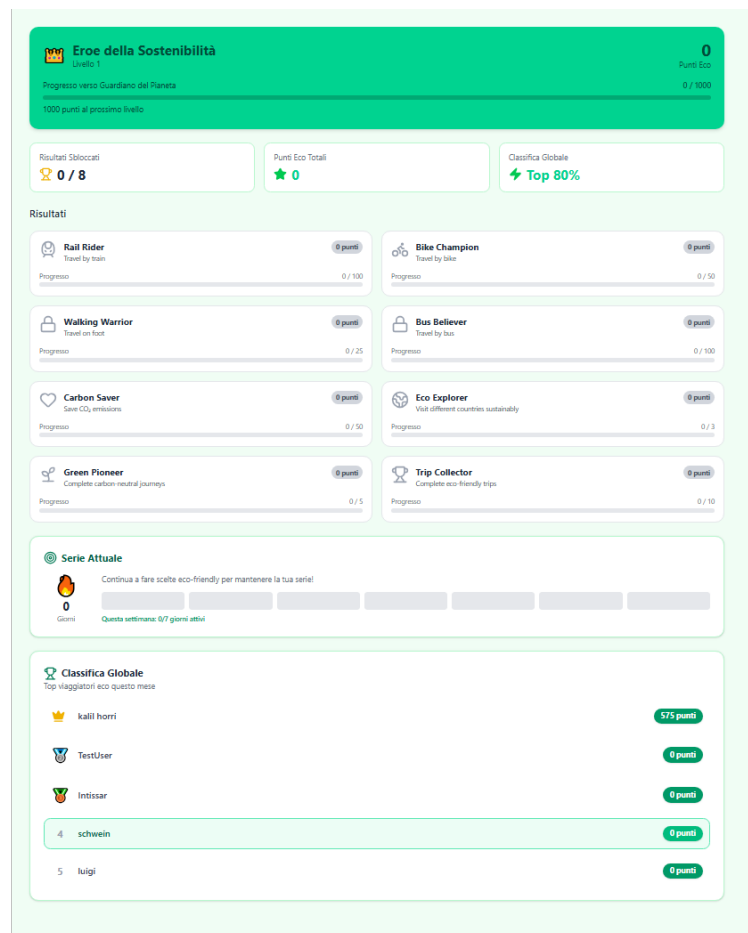


Figura 11: Gamification elements

4 Tecnologie

Di seguito sono illustrate le principali componenti del progetto e le librerie esterne utilizzate.

4.1 Pinia

Per la gestione dello stato globale dell'applicazione è stata usata **Pinia**, una libreria che facilita la condivisione dei stati quando si cambia la pagina. Nel contesto di EcoGo, Pinia centralizza la logica di business lato client attraverso store dedicati:

- **Trips Store:** Mantiene lo stato dei viaggi pianificati e in corso, permettendo l'aggiornamento degli itinerari.
- **Rewards Store:** Gestisce la logica di gamification, aggiornando in tempo reale punti, badge e progressi nei livelli.

4.2 Mapbox GL

La componente visiva per la pianificazione dei viaggi è realizzata tramite **Mapbox GL JS**. Questa libreria offre tantissime funzionalità come directions, traffics, form di search, etc..., garantendo fluidità e personalizzazione stilistica. L'integrazione con i servizi di Mapbox copre diverse esigenze funzionali:

- **Rendering Mappa:** Visualizzazione di mappa con stili personalizzati.
- **Geocoding:** Conversione degli indirizzi e dei punti di interesse (POI) in coordinate geografiche.
- **Traffic API:** Mostrare il traffico sulla mappa
- **Directions API:** Calcolo dei percorsi.

4.3 Turf.js

Per avere animazione sulla visualizzazione degli itinerari, è stato integrato **Turf.js**, una libreria permette di eseguire operazioni complesse direttamente nel browser su dati GeoJSON.

4.4 Socket.io

Per implementare le funzionalità di notifiche push, è stata usata **Socket.io**, una libreria che abilita una comunicazione bidirezionale tra client e server, utilizzando il protocollo WebSocket. L'utilizzo di Socket.io ci ha permesso di:

- Inviare di notifiche push all'utente (es. "Hai sbloccato un nuovo badge!").
- Ricevere allerte meteo e informazione sull'affluenza turistica
- Avere una promemoria di viaggi in corso

5 Codice e Funzionalità

Il codice implementa diverse funzionalità chiave suddivise in moduli:

5.1 Gestione delle chiamate llm

```
export async function generateDailyTips() {
  const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash-lite" });

  const prompt = `
    Generate 6 unique, practical, and short sustainable travel tips.

    Return ONLY a raw JSON array (no markdown, no backticks).
    Each object must have these exact keys:
    - "text": The tip content (min12words, max 30 words).
    - "icon": A string keyword strictly chosen from this list: ["Globe", "
      Droplet", "Smartphone", "Luggage", "Plug", "Footprints", "Recycle", "
      ShoppingBag", "Leaf", "Sun", "Wind"].
  `;

  try {
    const result = await model.generateContent(prompt);
    const response = await result.response;
    const text = response.text();

    const cleanedText = text.replace(/```json|```/g, "").trim();

    return JSON.parse(cleanedText);
  } catch (error) {
    console.error("Gemini_API_Error:", error);
    return [];
  }
}
```


5.2 Gestione comunicazione real-time con Socket.io

- notification:new

```
/**
 * Socket.io event handlers for notifications
 * Handles real-time push notifications to connected clients
 */

export const setupNotificationSocket = (io) => {
  io.on("connection", (socket) => {
    console.log("Client_connected_for_notifications:", socket.id);

    // Client joins their personal notification room
    socket.on("join:notifications", (userId) => {
      socket.join(`user:${userId}`);
      console.log(` User ${userId} joined notification room`);
    });

    // Client leaves notification room
    socket.on("leave:notifications", (userId) => {
      socket.leave(`user:${userId}`);
      console.log(` User ${userId} left notification room`);
    });

    socket.on("disconnect", () => {
      console.log("Client_disconnected:", socket.id);
    });
  });
};

/**
 * Emit notification to specific user
 * @param {Object} io - Socket.io instance
 * @param {String} userId - User ID to send notification to
 * @param {Object} notification - Notification data
 */
export const emitNotification = (io, userId, notification) => {
  io.to(`user:${userId}`).emit("notification:new", {
    id: notification._id,
    type: notification.type,
    city: notification.city,
    message: notification.message,
    icon: notification.icon,
    isRead: notification.isRead,
    createdAt: notification.createdAt,
  });
  console.log(`Pushed notification to user ${userId}:`, notification.type);
};
```

- notification:weather

```

/**
 * Emit weather notification to specific user
 * @param {Object} io - Socket.io instance
 * @param {String} userId - User ID
 * @param {Object} notification - Weather notification data
 */
export const emitWeatherNotification = (io, userId, notification) => {
  io.to(`user:${userId}`).emit("notification:weather", {
    id: notification._id,
    type: notification.type,
    city: notification.city,
    message: notification.message,
    icon: notification.icon,
    weatherData: notification.weatherData,
    isRead: notification.isRead,
    createdAt: notification.createdAt,
  });
  console.log(
    `Pushed weather notification to user ${userId}:`,
    notification.city
  );
};

```

5.3 Gestione stato con Pinia

Sul frontend, Pinia è utilizzato per mantenere uno stato condiviso e reattivo tra le pagine(ad esempio tra plan e world, plan e discover).

```
export const useTripStore = defineStore("trip", () => {
  const currentTrip = ref({
    id: null,
    name: "",
    routes: [],
  });
  const destinationFromDiscoverToPlan = ref("");
  function setDestination(address) {
    destinationFromDiscoverToPlan.value = address;
  }
  function setTripToEdit(tripFromDashboard) {
    const convertedRoutes = tripFromDashboard.routes.map((route) => {
      let dateStr = "";
      let depTimeStr = "";
      let arrTimeStr = "";

      if (route.rawStartTime) {
        const start = new Date(route.rawStartTime);
        dateStr = start.toISOString().split("T")[0]; // "YYYY-MM-DD"
        depTimeStr = start.toTimeString().slice(0, 5); // "HH:mm"
      }
      if (route.rawEndTime) {
        const end = new Date(route.rawEndTime);
        arrTimeStr = end.toTimeString().slice(0, 5); // "HH:mm"
      }

      return {
        id: route.id,
        from: route.from,
        to: route.to,
        fromCoords: route.startCoords,
        toCoords: route.endCoords,
        type: route.type,

        date: dateStr,
        departureTime: depTimeStr || route.depTime,
        arrivalTime: arrTimeStr || route.arrTime,

        gate: route.gate,
        transportNumber: route.transportCode,
        cost: route.cost,
        co2: route.co2,
        time: route.duration,
        distance: route.distance,
        seat: route.seat,
        class: route.class,
        travelClass: route.class,
```

```
    };  
  });  
  
  currentTrip.value = {  
    ...tripFromDashboard,  
    routes: convertedRoutes,  
  };  
}  
  
return {  
  currentTrip,  
  setTripToEdit,  
  setDestination,  
  destinationFromDiscoverToPlan,  
};  
});
```

6 Test

I test di usabilità hanno accompagnato l'intero processo di sviluppo del sistema. Fin dalle prime fasi di prototipazione, gli utenti hanno avuto modo di utilizzare e valutare il sistema, continuando a farlo fino alla sua realizzazione finale. In particolare, sono stati effettuati test di usabilità con tre partecipanti, gli stessi coinvolti nelle interviste preliminari. Le sessioni di test si sono concentrate sui principali task di Discover, Pianificazione e Tracciamento. I feedback raccolti nel tempo si sono rivelati preziosi per individuare criticità, correggere eventuali problemi e apportare modifiche alle funzionalità, contribuendo così a migliorare progressivamente l'esperienza d'uso e a renderla il più possibile piacevole e intuitiva.

Task	Success	Time	Key Issue
1. Discover (AI)	100%	30s	None.
2. Plan (Map)	100%	95s	Zooming on map felt sensitive.
3. Track (Stats)	100%	45s	None.

Table 1: Task Performance Metrics

5

Task	Success	Time	Key Issue
1. Discover (AI)	100%	60s	None.
2. Plan (Map)	100%	50s	None.
3. Track (Stats)	100%	100s	Not very impressed by stats.

Table 2: Task Performance Metrics

Task	Success	Time	Key Issue
1. Discover (AI)	100%	10s	None.
2. Plan (Map)	100%	95s	Lagging.
3. Track (Stats)	50%	90s	First time didn't found dashboard.

Table 3: Task Performance Metrics

Figura 12: Risultati dei test di usabilità

6.1 Risultati e Miglioramenti

I test effettuati hanno evidenziato quanto segue:

- **AI Tips:** la funzionalità è stata valutata positivamente; risulta tuttavia opportuno introdurre un indicatore di caricamento durante l'attesa della risposta.
- **Mappa:** è necessario migliorare la gestione della sensibilità dello zoom per rendere l'interazione più controllata.
- **Grafici:** alcune visualizzazioni andrebbero semplificate al fine di migliorarne la leggibilità e la comprensione.

7 Deployment

Per l'avvio dell'applicazione è stato scelto Docker, integrato con Doppler per gestire le variabili di ambiente. In fase di sviluppo si usavano file locali `.env`, mentre successivamente i secret sono stati centralizzati nel cloud per semplificare la configurazione e aumentare la sicurezza. Ora basta richiedere il token di accesso perché Doppler inietti automaticamente i secret necessari all'interno dei container.

7.1 Installazione

Prerequisiti:

- Doppler CLI
- Docker Desktop
- Git bash
- Service Token da richiederci

Usare Git bash per:

```
git clone https://github.com/schwein69/Asw-Hci.git
cd to the root path (mevn-app)
export DOPPLER_TOKEN=YOUR_SERVICE_TOKEN
doppler run -- docker-compose up --build
```

7.2 Messa in funzione

Accesso ai servizi:

- Frontend : <http://localhost:8080>
- API: <http://localhost:4000>

Comandi utili Docker:

```
docker compose up
docker compose logs -f
docker compose down
```

8 Conclusioni

Lo sviluppo di questo progetto ha rappresentato un'importante opportunità per ampliare le competenze acquisite durante il corso, consentendo di applicare le conoscenze teoriche in un contesto reale. Un ruolo rilevante nella realizzazione della piattaforma è stato ricoperto dall'utilizzo di **Vue.js**, in combinazione con **Vite**. Questa tecnologia ha permesso una migliore strutturazione del codice grazie a un'architettura basata su componenti, favorendo l'acquisizione di familiarità con strumenti moderni, performanti e orientati allo sviluppo lato client. Inoltre, sono state impiegate ulteriori librerie moderne, tra cui **Socket.io** per la comunicazione in tempo reale e **Pinia** per la gestione dello stato applicativo. Nel complesso, l'esperienza è stata valutata dal team come altamente formativa. Un aspetto particolarmente apprezzato è stato il lavoro svolto nell'ambito della **HCI (Human-Computer Interaction)**: la progettazione dell'esperienza utente si è rivelata non solo fondamentale per l'efficacia di un progetto orientato alla sostenibilità, ma anche un'attività stimolante e coinvolgente per l'intero gruppo. Inoltre, l'approccio adottato si è dimostrato pienamente compatibile con il corso di **ASW**, garantendo un'elevata semplicità e comodità nella realizzazione del frontend.